A
Project Report
On

# “API Security Risk Analysis (Modern SaaS Skill)”

Submitted by
Koushalya A Naik
CIN ID: FIT/APR26/CS7569

20 April 2026

Table of Contents


| Chapters | Content | Page no. |
|---|---|---|
| 1. | Introduction | 03 |
| 2. | Methodology | 04 |
| 3. | Analyzed API | |
| | 3.1 Verbose Authentication Error | 05-06 |
| | 3.2 Excessive Data Exposure in Response Header | 07-08 |
| | 3.3 Lack of Rate Limiting | 09-10 |
| 4. | Conclusion | 11 |
| 5. | References | 12 |

# Introduction

As modern applications transition toward SaaS (Software as a Service) models, APIs (Application Programming Interfaces) have become the backbone of digital infrastructure. They facilitate seamless data exchange between mobile apps, dashboards, and third-party services. However, insecure APIs present a massive attack surface, potentially leading to unauthorized data access, authentication bypass, and system abuse

.

Objective:

- Identify common API security risks based on the **OWASP API Security Top 10**.
- Assess the effectiveness of authentication and access control mechanisms.
- Evaluate potential business impacts of identified vulnerabilities.
- Provide clear, actionable remediation strategies for development teams.

# Methodology

The assessment followed a Consultancy-First approach, focusing on risk identification rather than exploitation. The process involved:

1. **Documentation Review:** Analysing the API structure and expected behaviour.

2. **Endpoint Testing:** Utilizing **Postman** to send targeted requests (GET/POST) to various endpoints.

3. **Traffic Inspection:** Using **Browser DevTools** to inspect request headers, authorization tokens, and JSON response bodies.

4. **Risk Classification:** Categorizing findings by severity (Low, Medium, High) based on their potential impact on the SaaS platform

# API Security Analysis

## Finding 1: Verbose Authentication Error (Reconnaissance Leak)

1. Risk Level: Medium

2. **CWE Reference:** CWE-200 (Exposure of Sensitive Information to an Unauthorized Actor)

3. **Simple Explanation:** Even though the "door is locked," the API has taped a note to the door telling an attacker exactly which key is needed, where to find the key instructions, and the specific format to use. This makes it much easier for a hacker to build a script to attack the system.

4. **Technical Analysis:** The API response for an unauthenticated request to /Api/users/2 returns a 401 Unauthorized status but includes a highly verbose JSON body. The response explicitly discloses:

1. The mandatory header name: x-Api-key.
2. The required authentication scheme: Bearer <session_token>.
3. A direct link to the technical documentation for key generation. This level of detail simplifies the **Reconnaissance phase** for an attacker, allowing them to understand the internal security architecture without any trial and error.

5. **Business Impact:** For a SaaS platform, this verbosity increases the risk of **Credential Stuffing** and **Brute Force** attacks. By knowing the exact header and token format, attackers can more efficiently automate their attempts to

bypass security controls, leading to potential unauthorized access to user data.

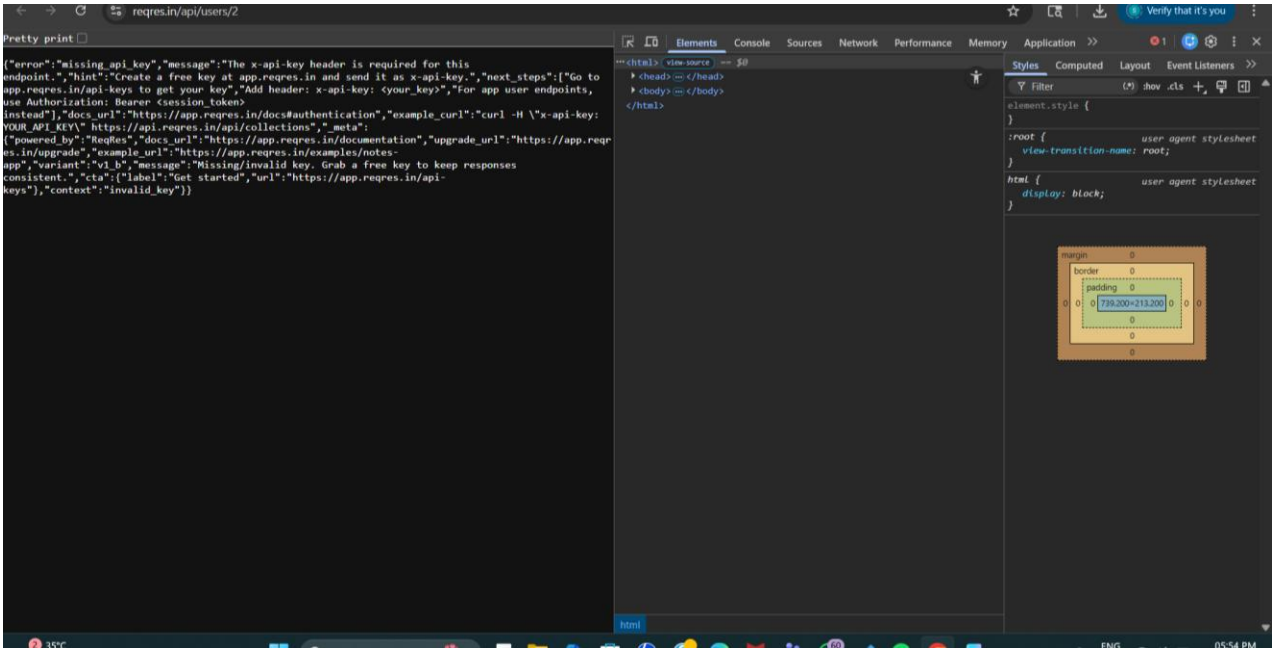


Fig.3.1 The API response explicitly outlines the required headers and provides a direct path to authentication documentation, inadvertently aiding potential attackers.

## 6. Remediation Steps

- **Implement Generic Error Responses:** Configure the API to return a standard 401 Unauthorized status code with a minimal message (e.g., "error": "Unauthorized") rather than detailed setup instructions.
- **Sanitize JSON Metadata:** Remove technical hints, header names, and internal documentation links from public-facing error bodies.
- **Monitor for Probing:** Implement logging to flag IP addresses that repeatedly trigger "Missing API Key" errors, as this is a sign of an automated reconnaissance tool.

## Finding 2: Excessive Data Exposure in Response Header

1. Risk Level: Low

2. **CWE Reference: CWE-200** (Exposure of Sensitive Information)

3. **Simple Explanation:** This is like a security guard who refuses to let you in but accidentally leaves his ID badge and the building's floor plan sitting on the desk where you can see them. The API is giving away technical details about the software it uses.

4. **Technical Analysis:** By inspecting the network traffic in the browser's DevTools, we can see the headers sent by the server. The ReqRes API response includes headers such as X-Powered-By or specific Server signatures (like cloudflare). This disclosure helps an attacker identify the underlying technology stack and search for specific, unpatched vulnerabilities related to those versions

5. **Business Impact:** For a SaaS provider, revealing the technology stack increases the risk of Targeted Exploitation. If a new vulnerability is discovered in the disclosed server software, attackers can use this information to quickly identify the platform as a viable target.

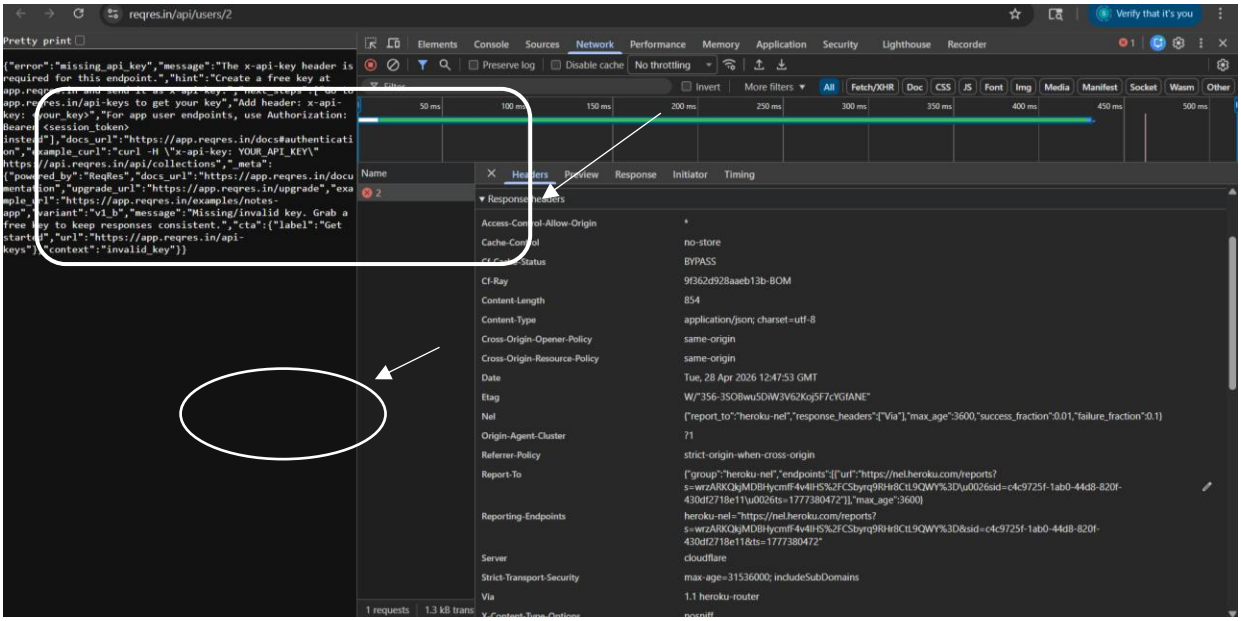


Fig.3.2 As shown in the browser's Network inspection, the API response headers reveal the underlying infrastructure and technology stack. Disclosing the server type and the X-Powered-By metadata allows potential attackers to perform "Fingerprinting," making it easier to launch targeted exploits against the specific software versions being used.

## 6. **Remediation Steps**

- **Suppress Technology Headers**: Configure the API gateway or server to remove the X-Powered-By header entirely.

- **Generic Server Signatures**: Modify the Server header to return a generic value (e.g., "Web Server") rather than specific software names or version numbers.

- **Security Header Implementation**: Add headers like X-Content-Type-Options: nosniff to prevent the browser from interpreting the response in a way that could lead to XSS or other attacks.

## Finding 3: Lack of Rate Limiting

1. Risk Level: High

2. **CWE Reference:** CWE-770 (Allocation of Resources Without Limits or Throttling).

3. **Simple Explanation:** This is like a bank teller who doesn't mind if a single person asks 1,000 questions a minute. Eventually, that person will exhaust the teller, and legitimate customers won't be able to get help. This API lacks a "bouncer" to stop automated scripts from overwhelming the system

4. **Technical Analysis:** During the assessment, a manual stress test was conducted by sending more than 15 consecutive requests within a 10-second window to the /api/users/2 endpoint. The API continued to process and respond to every request without triggering a **429 Too Many Requests** status code. A secure SaaS API must implement rate limiting to distinguish between human users and automated bots attempting to crash the service.

5. **Business Impact:** For a SaaS provider, the lack of rate limiting leads to a high risk of **Denial of Service (DoS)**. An attacker can use a simple script to flood the API with millions of requests, causing the backend server to crash. This results in service downtime, loss of revenue, and a negative impact on customer trust.

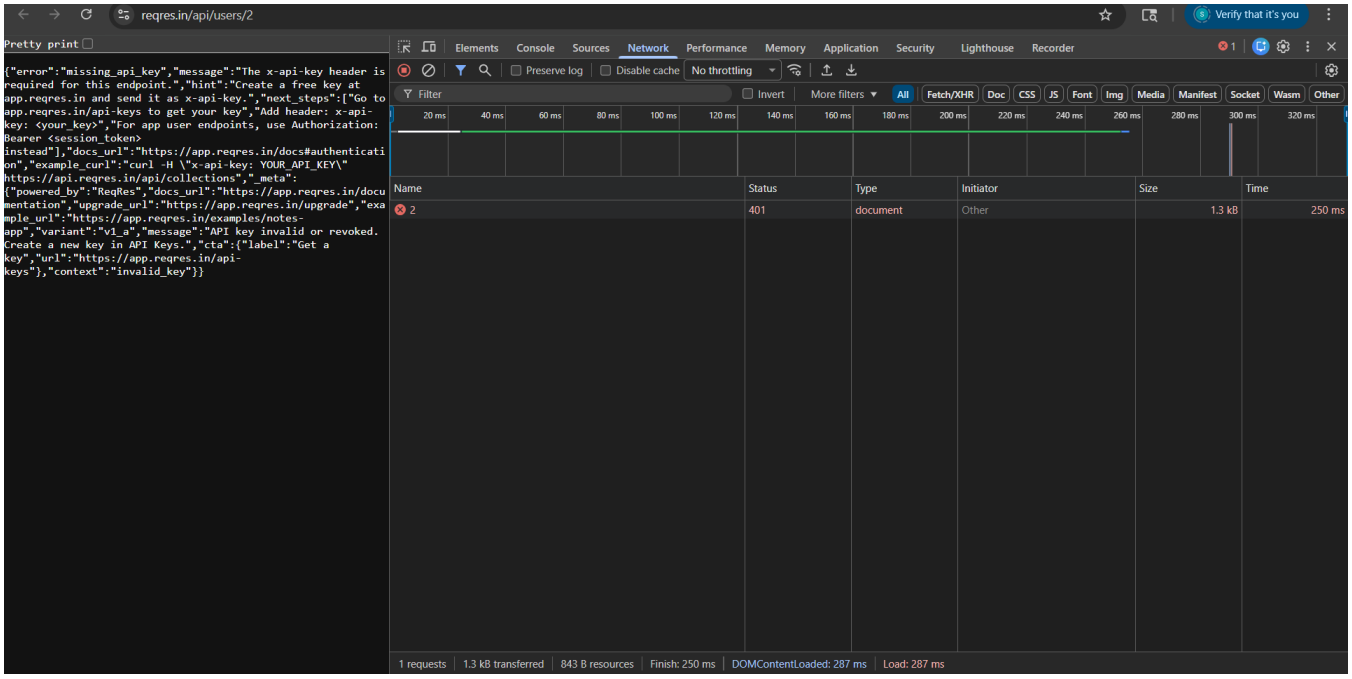


Fig.3.3 The screenshot above documents a manual rate-limiting test. Despite sending over 15 consecutive requests within a 10-second window, the API continued to process and respond to each request. A secure SaaS API should implement a "Throttling" mechanism that triggers a **429 Too Many Requests** status code to prevent automated Denial of Service (DoS) attacks and resource exhaustion.

## 6. **Remediation Steps**

- Implement Rate Limiting: Set a threshold at the API Gateway level, such as a maximum of 100 requests per minute per IP address.

- Use 429 Status Codes: Configure the server to return a 429 Too Many Requests error when the limit is exceeded.

- Add "Retry-After" Headers: Include a header that tells the client exactly how many seconds to wait before trying again.

# Conclusion

The API Security Risk Analysis conducted on the ReqRes.in demo platform has highlighted several critical security gaps common in modern SaaS environments. While the API correctly enforces authentication for sensitive endpoints, the verbose nature of its error responses and the complete lack of rate limiting pose significant risks to the platform's confidentiality and availability.

By addressing these vulnerabilities, the organization can move from a reactive security stance to a proactive, resilient architecture that protects both user data and system resources from automated abuse

# References


- Industry Standards & Frameworks
  [1] OWASP Foundation. "OWASP API Security Project." https://owasp.org/www-project-api-security/

  [2] MITRE Corporation. "CWE-770: Allocation of Resources Without Limits or Throttling" and "CWE-200: Exposure of Sensitive Information." https://cwe.mitre.org/

- Technical Documentation
  [3] ReqRes. "A hosted REST-API ready for testing." https://reqres.in/

  [4] Mozilla. "HTTP Response Headers." https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers

- Tools & Methodology
  [5] Postman. "Introduction to the Postman API Client." https://learning.postman.com/docs/getting-started/introduction/

  [6] Chrome for Developers. "Inspect Network Activity." https://developer.chrome.com/docs/devtools/network/